ML-based Deep Bitstream Inspection for Serverless FPGAs

SANTIAGO DE HEREDIA, Systems Group, ETH Zurich, Switzerland
MAXIMILIAN J. HEER, Systems Group, ETH Zurich, Switzerland
GUSTAVO ALONSO, Systems Group, ETH Zurich, Switzerland

Cloud and serverless FPGA platforms expose reconfigurable fabric to user-submitted designs, creating risks from malicious circuits such as ring oscillators. Existing machine-learning approaches typically inspect bitstreams offline with additional hardware, including GPUs. In moving towards an accelerator model where the FPGA functions as a completely standalone unit, we instead study an on-FPGA checker that analyzes incoming partial bitstreams before reconfiguration. We generate benign accelerator applications and malicious bitstreams, implement an end-to-end machine learning preprocessing and inference pipeline, and synthesize the model directly into the FPGA using hls4ml. Our selected model architecture achieves 88.5% F1 and 95.1% ROC AUC. On an Alveo U55C, our checker is capable of an end-to-end preprocessing latency of 1.118–4.394 ms from host invocation across a range of bitstream sizes, ranging from 5.28 to 53.10 MiB. Its routed wrapper uses 4.69% of device LUTs, 2.70% of BRAM, 7.18% of DSPs, and no URAM. These results demonstrate the feasibility of an on-FPGA checker for detecting malicious bitstreams before partial reconfiguration.

CCS Concepts: • **Security and privacy** → **Systems security**; • **Computer systems organization** → *Cloud computing*; • **Computing methodologies** → *Machine learning*; Neural networks.

Additional Key Words and Phrases: cloud FPGA security, bitstream inspection, partial bitsreams, malicious circuit detection, serverless FPGAs, partial reconfiguration, convolutional neural networks, ML for systems, hls4ml

ACM Reference Format:
Santiago de Heredia, Maximilian J. Heer, and Gustavo Alonso. 2026. ML-based Deep Bitstream Inspection for Serverless FPGAs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Field-programmable gate arrays (FPGAs) are increasingly used as datacenter accelerators because they combine programmability with high performance and energy efficiency [3, 5]. Cloud and serverless FPGA platforms extend this model by allowing users to deploy hardware applications on shared infrastructure. Partial reconfiguration (PR) makes this practical by loading user logic into a reconfigurable region while the trusted FPGA shell remains active [13, 20].

This deployment model exposes shared FPGA resources to untrusted user logic. Malicious circuits such as ring-oscillator (RO) arrays can generate sustained switching activity, causing voltage drops, timing faults, and denial of service [4, 10, 14, 19]. Demonstrated attacks on cloud FPGA infrastructure therefore motivate inspecting user bitstreams before partial reconfiguration [14].

Authors' Contact Information: Santiago de Heredia, sdeheredia@ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland; Maximilian J. Heer, maximilian.heer@inf.ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland; Gustavo Alonso, alonso@inf.ethz.ch, Systems Group, ETH Zurich, Zurich, Switzerland.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Prior work has shown that machine-learning models can identify malicious designs from bitstream-derived representations without fully reconstructing their circuit semantics [6, 9, 18, 21]. However, these approaches are commonly evaluated as software-based and offline classification pipelines, including the need for additional hardware asides from the FPGA, especially GPUs for inference[7]. A serverless FPGA platform introduces additional requirements and goals that make these solutions undesired:

- **Latency:** Bitstream inspection should not contribute significantly to the latency of deploying said bitstream.
- **Economics and Hardware:** Ideally, the accelerator system should be kept simple and inexpensive, avoiding the need for additional servers and / or specialized hardware.

Offline approaches trade complexity at the expense of blocking deployment of the bitstream until inspection has taken place in a separate process. Moreover, reserving additional specialized hardware for (online) bitstream inspection and inference (especially with GPUs) make this option less attractive for commercial ventures like AWS F2**[TODO]**.

We investigate an *on-FPGA checker* placed before partial reconfiguration of the user bitstream. The checker preprocesses the raw bytes of the incoming bitstream, executes a quantized convolutional neural network (CNN), and returns a suspiciousness score to the deployment logic. We study this design in the context of Coyote, a datacenter-grade FPGA shell supporting PR and multitenancy [20]. We generate benign Coyote application bitstreams and RO-based malicious bitstreams, train compact CNNs, and synthesize and optimize selected models using hls4ml [1, 8].

The resulting checker provides useful detection at millisecond-scale latency while leaving sufficient resources for the Coyote shell and user logic.

This work makes the following four contributions:

- We construct a dataset of benign Coyote partial bitstreams and RO-based malicious bitstreams across multiple floorplans and payload footprints.
- We develop an on-FPGA inspection pipeline combining deterministic byte-stream preprocessing, quantized CNN inference, and hls4ml synthesis.
- We evaluate detection quality, latency, and routed FPGA resource overhead together, identifying a practical deployment point for pre-reconfiguration inspection.
- We compare our on-FPGA checker system against an GPU checker integrated with the FPGA, demonstrating lower-or-parity end-to-end latency compared to the baseline.

2 Background, Threat Model, and Related Work

2.1 Deployment Setting and Threat Model

Serverless FPGA platforms allow users to deploy hardware accelerators on shared reconfigurable infrastructure. Partial reconfiguration (PR) supports this model by dividing the FPGA into a trusted static shell and one or more reconfigurable regions containing user logic. The shell remains active while individual user regions are reconfigured, allowing the platform to retain control over host communication, memory, networking, and runtime services [13, 20].

This separation between a static shell and a reconfigurable region creates a security boundary. A user-submitted partial bitstream is an untrusted input that directly determines the circuit instantiated inside a reconfigurable region. Malicious circuits can exploit physical resources shared across the device, including power delivery, thermal headroom, and timing margins. We focus on ring-oscillator (RO) payloads, which produce sustained switching activity and can induce voltage drops, timing faults, or denial of service when instantiated at sufficient scale [4, 10, 14, 19].

Table 1. Comparison of malicious-bitstream detection approaches across deployment and implementation dimensions.

Approach	Detection basis	Deployment		Properties		Target
		Deployment integrated	On-FPGA	No reverse engineering	Datacenter target	
Platform checks	Provider-defined rules	✓	×	✓	✓	Cloud deployment
Structural scanning	Circuit structures	×	×	×	×	Device-specific FPGA
Offline ML	Learned features	×	×	✓	×	Host software
Embedded ML	Byte-frequency features	×	×	✓	×	PYNQ-Z1
This work	**Learned byte patterns**	**Proposed**	✓	✓	✓	**Coyote/U55C**

Legend: ✓indicates that a property is demonstrated, while ×indicates that it is not demonstrated. "Proposed" denotes integration designed as part of this work but not yet connected to the complete Coyote control path.

We consider an attacker who can submit a syntactically valid partial bitstream for a user region. The bitstream contains an RO-based payload intended to pass existing platform checks and stress shared physical resources after configuration.

The defender places an on-FPGA checker before PR. The checker inspects the bitstream that would subsequently be provided to the PR controller. It produces a maliciousness score that the trusted deployment logic converts into an allow or reject decision. A true positive blocks and logs a suspicious bitstream. A false negative would allow a malicious bitstream to be configured, while a false positive would reject a benign accelerator applicator.

2.2 Existing Defenses and Research Gap

Existing defenses against malicious FPGA designs broadly fall into three groups. Table 1 summarizes their relationship to this work.

Platform and vendor checks are already part of practical cloud-FPGA deployment flows, but attacks have shown that syntactically valid power-wasting circuits can pass such checks [14]. Structural approaches instead search for specific circuit patterns, including self-oscillators [15, 24]. These methods provide direct evidence of suspicious structures, but often depend on device-specific architectural knowledge or bitstream reverse engineering.

Machine-learning approaches avoid explicit reconstruction of the implemented circuit. Prior work has classified malicious designs using features derived from FPGA bitstreams, including artificial neural networks, CNNs, and other supervised classifiers [6, 9, 18, 21]. These studies establish that bitstream-level representations contain learnable information. However, they primarily evaluate detection as an offline software task and do not jointly consider placement in a datacenter FPGA deployment path, FPGA resource cost, and inspection latency.

Embedded approaches demonstrate that bitstream inspection can execute locally on a PYNQ-Z1 platform without external computation [23]. BLADEI further investigates deployment latency and proposes hardware-accelerated preprocessing, while retaining classification on the embedded ARM processor [22]. Serverless FPGA deployment requires balancing inspection latency with hardware resource usage. Screening delays contribute directly to deployment time, while checker

resources compete with the platform shell and user accelerators. Our work places both preprocessing and quantized model inference in programmable logic with hls4ml, and optimizes their execution through streaming and resource tuning. We evaluate the resulting checker on Coyote and the Alveo U55C, measuring detection quality, inspection latency, and FPGA resource cost together.

This leaves three feasibility questions:

(1) **Can the checker retain useful detection quality, particularly a sufficiently low false-negative rate?**
(2) **Can the checker fit alongside the Coyote shell and user logic while adding acceptable inspection latency?**
(3) **Can preprocessing, inference, and decision logic be placed on the FPGA and deployment path, without a separate host-side classification stage?**

The following sections evaluate these questions through dataset construction, model training, hardware synthesis, and a proposed Coyote integration.

3 Dataset and Preprocessing

3.1 Dataset

We construct a binary classification dataset of benign Coyote partial bitstreams and malicious RO-based partial bitstreams generated through the Coyote partial-reconfiguration flow. The benign class contains eight Coyote applications spanning HLS, cryptographic, communication, and compute examples. We build these applications across multiple Coyote floorplans, yielding 221 benign partial bitstreams.

Malicious samples use the RO-based designs of Mahmoud et al. [16], instantiated at different payload sizes and floorplans. The corpus contains 225 malicious partial bitstreams.

3.2 Preprocessing

We uniformly downsample each raw partial bitstream to $H \cdot W$ bytes, using evenly spaced positions from the first byte to the last and rounding each position down to an integer index. Shorter bitstreams are padded with zero bytes to reach the same length. We then invert and normalize each byte value b as $(255 - b)/255$ and reshape the sequence into an $H \times W \times 1$ grayscale tensor. Padding therefore produces pixels with value 1.

Our final checker uses a 256×256 input, corresponding to 65,536 sampled bytes per partial bitstream. We evaluate additional input resolutions during the model design-space exploration in Section 4.

4 ML-Based Bitstream Inspection

We use convolutional neural networks (CNNs) to classify the fixed-size bitstream representations introduced in Section 3. Prior work has shown that bitstream-derived representations contain learnable structure and can be used to distinguish benign and malicious FPGA designs [6, 9, 18, 21]. CNNs are also well suited to our deployment goal because quantized models can be translated into FPGA circuits using tools such as hls4ml [1, 8].

4.1 CNN Architecture

The selected checker uses a seven-stage convolutional architecture operating on a 256×256 byte-image tensor. Each stage consists of a quantized convolution, quantized ReLU activation, and max pooling. Channel width increases from 8 filters in the first stage to 32 filters in the later stages.

Table 2. Seven-stage quantized CNN architecture used by the on-FPGA checker. All convolutional stages use quantized weights and ReLU activations.

Stage	Filters	Kernel	Stride	Pooling
Conv 0	8	5×5	2×2	2×2
Conv 1	16	3×3	1×1	2×2
Conv 2	24	3×3	1×1	2×2
Conv 3	24	3×3	1×1	2×2
Conv 4	32	3×3	1×1	2×2
Conv 5	32	3×3	1×1	2×2
Conv 6	32	3×3	1×1	2×2
Output	1	–	–	AvgPool

A final average-pooling operation reduces the remaining feature map before a single quantized dense output produces the classification logit.

Table 2 summarizes the architecture. We use quantized weights and activations and do not use batch normalization. The output logit is converted to a maliciousness probability using a sigmoid; the decision threshold is applied later by the deployment logic.

4.2 Training

Models are implemented in TensorFlow [2] using QKeras [11]. We use quantization-aware training [11], so that the model is trained under the same fixed-precision behavior used during hardware generation. Training minimizes binary cross-entropy from logits. We augment the training data with horizontal and vertical flips and use a balanced production training set containing equal numbers of benign and malicious samples.

During model development, we use stratified cross-validation to compare model configurations [12]. After selecting the final architecture, we train a production model separately and evaluate it on the held-out test set described in Section 6.1.

4.3 Hardware-Aware Design-Space Exploration

Model selection considers hardware feasibility in addition to software classification quality. Rather than exhaustively evaluating every combination of parameters, we explore the design space in stages. We first vary input resolution and CNN depth to determine how model capacity and retained bitstream information affect classification quality. We then evaluate quantization and pruning as model-compression techniques before passing selected networks to the hardware synthesis flow described in Section 5.

The principal model-level dimensions are therefore input resolution, network depth, and numerical precision. Hardware parallelism is explored separately after conversion to hls4ml by varying operation reuse and per-layer configuration.

Figure 1 summarizes the software exploration. Detection quality generally improves with input resolution and CNN depth, with smaller gains at the largest configurations. These are pooled-validation results used for model selection, distinct from the held-out test results in Section 6.2. Considering detection quality and hardware cost, we select the 256×256, seven-stage quantized CNN. This is the model synthesized and evaluated in the remainder of the paper.

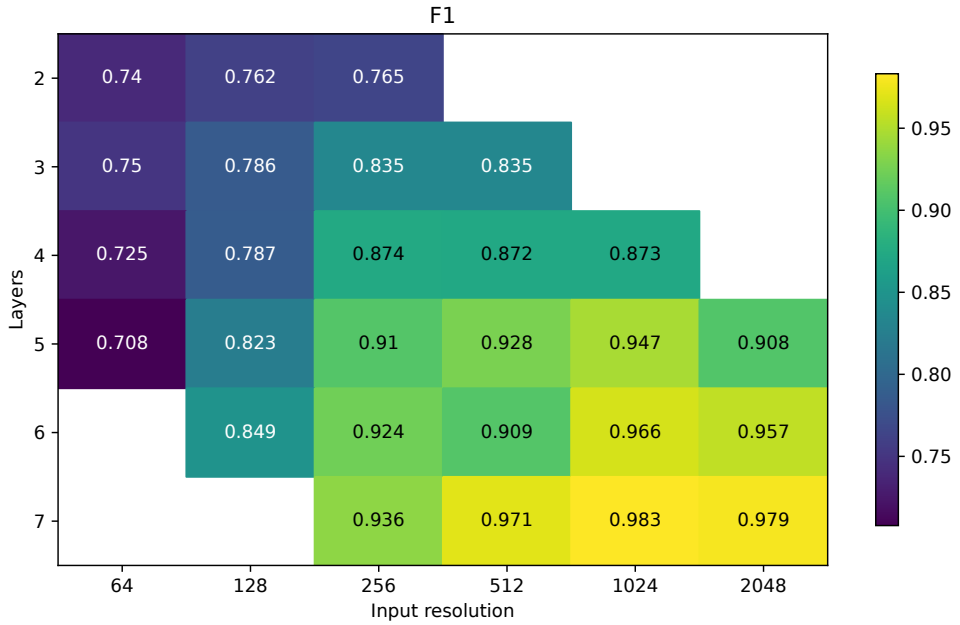


Fig. 1. Pooled-validation F1 across input resolution and CNN depth. This software exploration informs selection of the 256 × 256, seven-stage checker.

5 On-FPGA Checker

We next map the trained bitstream classifier into FPGA logic and consider how the resulting checker can be placed in the Coyote deployment path. The hardware design consists of two components: deterministic byte-stream preprocessing and the quantized CNN classifier. Importantly, both execute on the FPGA, allowing inspection to take place without a separate host-side classification stage.

5.1 Hardware Generation

We convert the trained QKeras CNN to hardware using hls4ml [1, 8]. hls4ml translates the quantized network into an HLS implementation, which is first evaluated through C simulation and subsequently synthesized to RTL. The final models use quantization-aware training, so the weight and activation precision used during training already reflects the quantized representation targeted by the generated hardware.

Before synthesis, we compare the hls4ml C-simulation output against the corresponding QKeras model. We require matching classifications on the validation inputs and inspect output deviations before synthesis. This provides a parity check between the trained model and its fixed-point hardware implementation.

The preprocessing operation from Section 3.2 is implemented separately in HLS C++ and lowered to RTL. It operates directly on the incoming bitstream bytes and reproduces the deterministic sampling, padding, inversion, and normalization used during training. The board latency reported in Section 6.3 measures host invocation to verdict availability inside the FPGA, including input transport and overlapping preprocessing and inference.

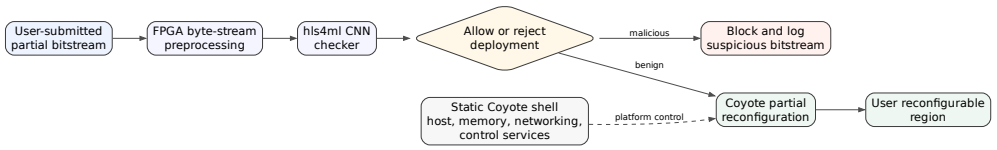


Fig. 2. Proposed integration of the on-FPGA checker into the Coyote deployment path. FPGA-side preprocessing and CNN inference inspect an incoming partial bitstream before it is passed to the partial-reconfiguration controller.

5.2 Hardware Optimization

Once the trained network is fixed, hls4ml exposes a hardware design space that trades resource usage against inference latency. The main parameter is the *reuse factor*, which controls how aggressively arithmetic units are shared across operations. Lower reuse increases parallelism and reduces latency at the cost of additional hardware, while higher reuse reduces resource use but increases execution time.

We initially evaluate resource-oriented hls4ml configurations and sweep reuse factors for the selected CNN architectures. A single global reuse setting, however, provides only a coarse trade-off because convolutional stages differ substantially in their operation counts and hardware cost. We therefore also apply manual per-layer reuse settings, assigning greater parallelism to latency-dominant layers while retaining more sharing elsewhere.

Quantization and pruning are additionally explored as hardware-aware model transformations. We train the quantized checker through QKeras quantization-aware training. For each transformed model, we retain a configuration only when software detection quality and hls4ml prediction parity remain acceptable. With the selected weights fixed, we then overlap preprocessing with inference and tune FIFO depths and per-layer reuse through hls4ml configuration. Section 6.3 evaluates the resulting latency and routed FPGA resources.

5.3 Proposed Coyote Integration

Coyote separates trusted platform services from partially reconfigurable user regions, providing a natural location for bitstream inspection before PR [20]. We propose placing the on-FPGA checker in this trusted deployment path, as illustrated in Figure 2.

An incoming user partial bitstream is first processed by the FPGA-side preprocessing logic and passed to the synthesized CNN. The CNN produces a maliciousness score, which trusted deployment logic compares against a configured threshold. Bitstreams classified as suspicious are rejected, while accepted bitstreams are allowed to proceed to the Coyote partial-reconfiguration controller.

The integration shown in Figure 2 is a proposed deployment architecture rather than a completed end-to-end Coyote implementation. This work implements and evaluates the FPGA preprocessing and CNN checker components, and connection to Coyote's PR controller and the associated control-plane logic remain future work.

6 Evaluation

We evaluate whether the proposed on-FPGA checker provides useful detection quality while remaining practical for deployment alongside the Coyote shell and user logic. In particular, we answer four questions:

(1) **Detection quality.** How accurately does the checker distinguish benign Coyote partial bitstreams from bitstreams containing ring-oscillator (RO) payloads?

(2) **Hardware feasibility.** What latency and resource overhead does the checker introduce on the target FPGA?
(3) **GPU comparison.** How does the FPGA checker's verdict latency compare with GPU inference?
(4) **Deployment-path overhead.** How significant is the added inspection latency relative to partial reconfiguration?

We evaluate the selected 256 × 256, seven-stage CNN. Section 6.3 describes the synthesized checker and our optimizations. Section 6.4 compares its verdict latency with GPU inference, and Section 6.5 places the inspection cost in deployment context.

6.1 Experimental Setup

We evaluate the checker on the held-out test set defined in Section 4. The positive class is the malicious RO class: a false negative therefore represents a malicious bitstream accepted by the checker, while a false positive represents a benign accelerator rejected by the platform.

Predictions use a probability threshold of 0.5, with higher values indicating greater confidence that a bitstream is malicious. We report accuracy, F1, true-positive rate (TPR), false-negative rate (FNR), false-positive rate (FPR), and ROC AUC.

Benchmark platform. Figure 3 shows the measured setup. An AMD Alveo U55C connects to the host through PCIe Gen3 ×16, while an AMD Instinct MI210 GPU connects through PCIe Gen4 ×16. The benchmark process is pinned to CPU64 on NUMA node 1, which also hosts the FPGA. The GPU is attached to NUMA node 0. The FPGA checker runs at 250 MHz. Resource counts are attributed to the routed user wrapper, including its interfaces and instrumentation, and percentages use the complete U55C capacity as their denominator. Shell resources are excluded from this attribution.

Compared implementations. `FPGA_HLS4ML` denotes our resource-optimized quantized checker. `GPU_HOST` performs inference on the GPU and relays its verdict to the FPGA through the host. Section 6.4 describes the GPU implementation and comparison.

Workloads and timing. The main comparison uses eight real bitstreams ranging from 5.28 to 53.10 MiB. Each input is already prepared in host memory when timing starts. We report the median and 95th percentile (p95) of 200 measured calls per input and implementation, excluding compilation and warmup. A synthetic 4 MiB input illustrates execution overlap.

The primary interval starts at host invocation and ends when the classification result is accepted inside the FPGA. This interval includes transport, preprocessing, inference, and verdict delivery as they occur along each path. We also record the time it takes for the full bitstream to reach the FPGA, FPGA operation completion, and the time for host observation. We exclude the time it takes to read the file from disk.

6.2 Detection Quality

Table 3 reports the detection quality of the selected checker on the held-out test set. The model reaches 87.7% accuracy and an F1 score of 88.5%. It detects 92.0% of the malicious bitstreams, corresponding to an FNR of 8.0%. Its ROC AUC of 95.1% indicates that the model provides substantial separation between the benign and malicious classes across possible decision thresholds.

The security-relevant error is the FNR because each false negative represents an malicious bitstream that would pass the checker. At the default threshold, the model misses 8.0% of malicious test samples. The corresponding FPR is 16.9%, meaning that the same operating point rejects a non-negligible fraction of benign applications.

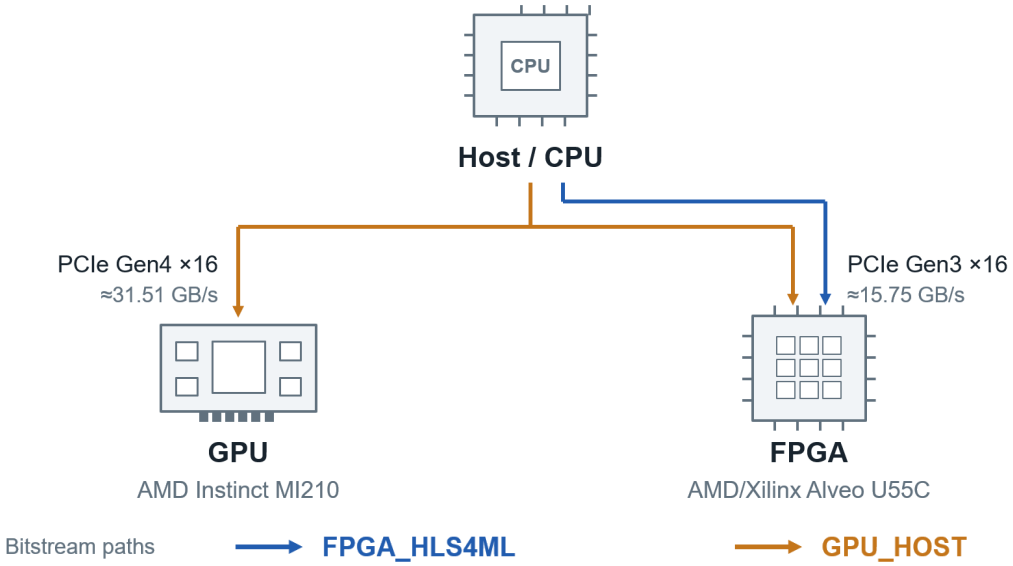


Fig. 3. Benchmark setup and input paths. FPGA_HLS4ML sends the bitstream to the U55C for preprocessing and inference. GPU_HOST sends independent copies to the GPU and FPGA; the host subsequently relays the GPU verdict to the FPGA. Link bandwidth labels are theoretical one-direction ceilings before transaction overhead, not measured payload rates.

Table 3. Detection quality of the selected 256^2-input, seven-layer checker on the held-out test set.

Accuracy	F1	TPR	FNR	FPR	ROC AUC
87.7%	88.5%	92.0%	8.0%	16.9%	95.1%

This trade-off depends on the decision threshold. Reducing the threshold would reject more malicious bitstreams, but would also block more benign accelerators. Increasing it would reduce the operational cost of false positives while admitting more malicious designs. We use threshold 0.5 as a fixed and reproducible threshold throughout the evaluation. A production deployment could instead select the threshold according to an explicit platform policy, such as minimizing the FNR subject to a maximum acceptable FPR.

6.3 Synthesized Checker and Optimizations

We synthesize the selected 256 × 256, seven-stage quantized model using hls4ml and place and route its wrapper for the U55C. The final implementation combines our streaming optimization with FIFO and arithmetic-reuse tuning. These optimizations leave the trained model weights unchanged.

Resource use. Table 4 reports the final routed wrapper resources. The checker occupies 2.70% of device BRAM and 7.18% of DSPs, while using 4.69% of LUTs and no URAM.

Our streaming optimization. Our initial implementation buffered the complete preprocessed image before starting the CNN. We optimized this path by connecting the preprocessor and CNN through an HLS stream in a DATAFLOW region and launching the CNN once the first valid pixel

Table 4. Resources of the synthesized resource-optimized streaming checker on the Alveo U55C. Counts cover the routed user wrapper, including interfaces and instrumentation. The Coyote shell is excluded, and percentages are relative to device capacity.

Resource	Count	U55C utilization
LUT	61,141	4.69%
BRAM	54.5 tiles	2.70%
DSP	648	7.18%
URAM	0 blocks	0%

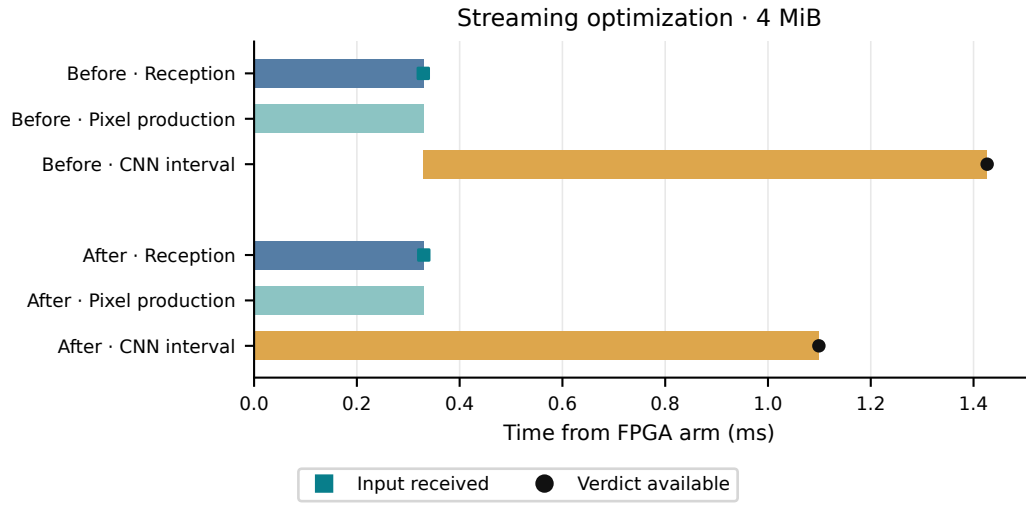


Fig. 4. Execution before and after our streaming optimization for a 4 MiB input, using representative trials nearest median verdict latency. Before the optimization, the complete preprocessed image is buffered. After it, CNN execution begins while pixels are still being produced. Squares mark full-input arrival and circles mark verdict availability. The CNN interval includes stalls waiting for pixels. These measurements isolate the streaming change before subsequent FIFO and reuse tuning.

is available. This allows reception, pixel production, and inference to overlap. Figure 4 illustrates this change for a 4 MiB input. The median invocation-to-verdict latency falls from 1.429 ms before this optimization to 1.102 ms after it. The figure uses the FPGA arm event as its origin, so its endpoints exclude the short host-invocation-to-arm interval.

We then reduce memory and arithmetic costs through native hls4ml settings. FIFO candidates are derived using native FIFO optimization with a profiling depth of 1,024 entries, and their behavior is checked in board benchmarks. For DSPs, we increase reuse factors in later convolutional stages, setting `conv3`/`conv4` to 72 and `conv5`/`conv6` to 288. The first two convolutional stages retain reuse factor one. These changes use completely native hls4ml configuration, and we do not do custom optimizations to the generated HLS kernels.

6.4 GPU Inference and Comparison

We compare the synthesized checker with GPU inference using the platform, workloads, and timing protocol in Section 6.1. Both paths are measured until the verdict is accepted inside the FPGA.

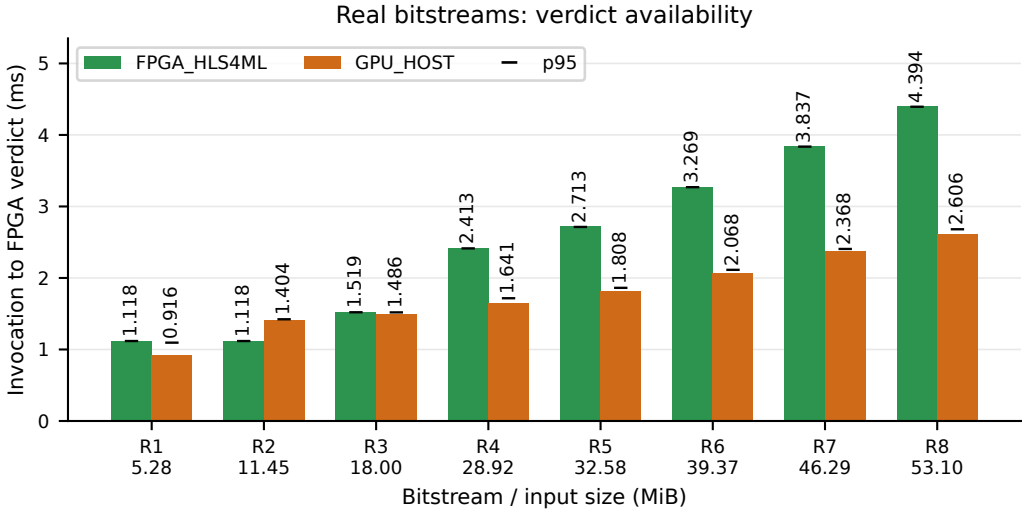


Fig. 5. Prepared-host-input latency to verdict acceptance inside the FPGA for eight real bitstreams. Bars show medians, numerical labels give milliseconds, and black ticks mark p95 across 200 calls per input and implementation. FPGA_HLS4ML uses the resource-optimized quantized checker; GPU_HOST uses float inference optimized with XLA compilation and cached ConcreteFunction calls. File reading and input preparation are excluded.

GPU implementation. GPU_HOST uses the corresponding non-quantized model with batch size one. This GPU baseline is also optimized with corresponding standard native configurations - its TensorFlow runtime combines preprocessing and inference in an XLA-compiled graph and invokes a cached ConcreteFunction for each input size. This comparison evaluates the quantized FPGA checker and the float GPU implementation, including their different numerical precisions.

The host submits independent input copies to the GPU and FPGA, as shown in Figure 3. Once the GPU input is ready, the runtime executes preprocessing and inference, synchronizes the result, and relays the verdict through the host to the FPGA. The FPGA records when it accepts that verdict, separately from when it receives the complete input bitstream.

Latency comparison. Figure 5 compares the final FPGA checker with GPU_HOST over the eight real bitstreams. FPGA median latency ranges from 1.118 to 4.394 ms. At R2 (11.45 MiB), the FPGA is 1.26× faster than the GPU. GPU_HOST is faster for the other seven inputs and is 1.69× faster at R8 (53.10 MiB). The comparison consequently does not show a uniform FPGA latency advantage.

The input paths differ as well as the compute implementations. GPU_HOST can copy the bitstream to the GPU through its faster PCIe link independently of the FPGA copy. Its verdict can be accepted inside the FPGA before the FPGA has received the entire bitstream. In contrast, the streaming checker consumes pixels generated from the incoming FPGA stream. At larger sizes, its verdict closely follows full-input arrival. Input transport therefore contributes to the observed difference, although nominal PCIe link rates alone do not isolate all host, transfer, and runtime costs.

Execution breakdown. Figure 6 shows representative 4 MiB trials for both implementations on a common invocation-relative time scale. The FPGA overlaps reception, pixel production, and CNN

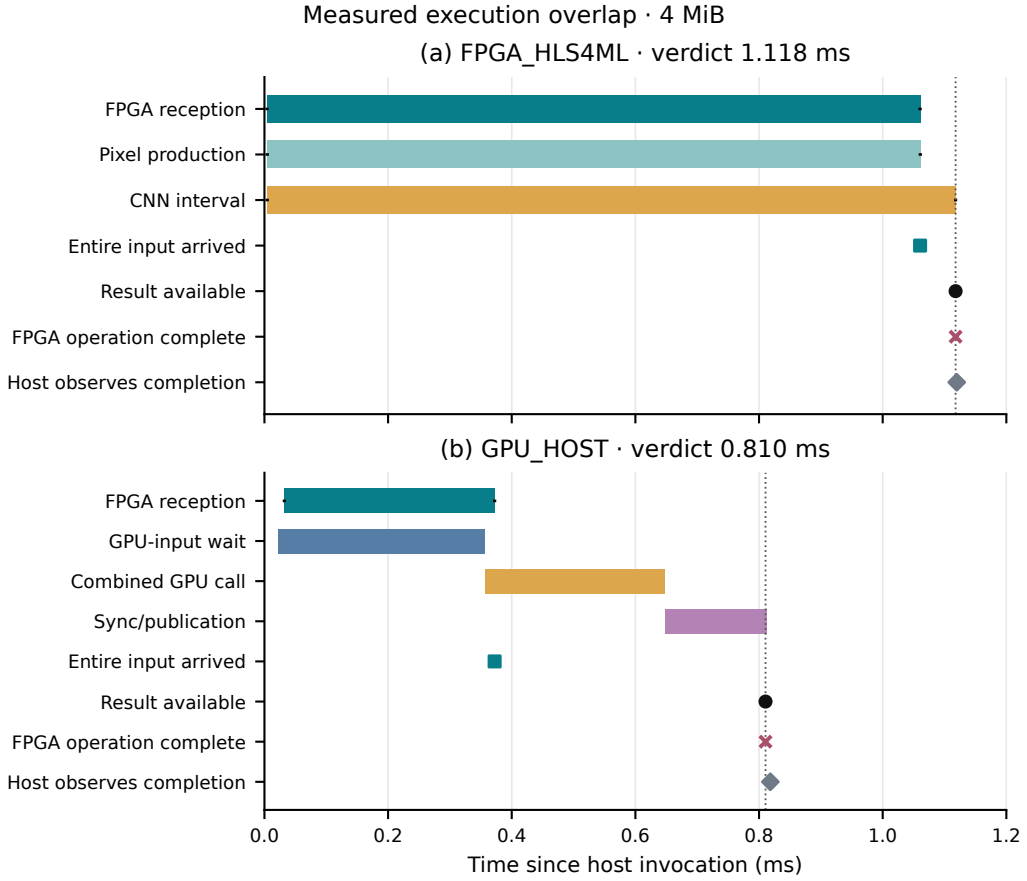


Fig. 6. Measured execution overlap and FPGA result availability for a 4 MiB input. Panels show the actual trials nearest median verdict latency (FPGA trial 95 and GPU trial 1382), with a shared horizontal scale. Squares mark full-input arrival, circles mark verdict acceptance, crosses mark FPGA operation completion, and diamonds mark host observation. Dotted lines mark verdict acceptance. Small black endpoint marks show clock-alignment bounds, not confidence intervals. The FPGA CNN interval includes input stalls; GPU intervals include framework and synchronization work.

execution, accepting its verdict at 1.118 ms. Its reception takes longer than the FPGA reception in GPU_HOST because the checker can backpressure the incoming stream.

GPU_HOST waits for GPU input readiness, calls the combined preprocessing and inference graph, and synchronizes and publishes its result through the host. It accepts the verdict at 0.810 ms in the illustrated trial. The GPU call and synchronization lanes are host-observed intervals, not isolated kernel times. In particular, publication can remain active briefly after the FPGA accepts the verdict. The final markers distinguish result availability from operation completion and the host's subsequent observation. Durations on parallel lanes must not be added.

Table 5. Contextual estimates obtained by adding 2.713 ms of measured inspection latency to published reference times, assuming no overlap. These are not measurements of an integrated deployment. The ICAP row is a throughput-based lower bound, excluding fixed controller and driver overhead.

Reference	Baseline	With inspection	Increase
F3 warm invocation with reconfiguration [17]	141 ms	143.71 ms	1.92%
Coyote v2 shell PR #2 [20]	72.3 ms	75.01 ms	3.75%
Coyote v2 shell PR #3 [20]	85.5 ms	88.21 ms	3.17%
Coyote v2 ICAP at 32.58 MiB [20]	≈42.70 ms	≈45.42 ms	≈6.35%

6.5 Inspection Latency in Deployment Context

We compare inspection latency with published serverless invocation and PR times. The median size among our eight real benchmark bitstreams is 30.75 MiB. We use the next measured size, 32.58 MiB (34,162,708 bytes), for which the final checker has a median invocation-to-verdict latency of 2.713 ms. Table 5 adds this interval to each reference, assuming inspection and the reference operation do not overlap.

F3 reports 141 ms for a warm SHA3 invocation that includes reconfiguration [17, Table 4]. Adding the selected inspection interval gives an estimated 1.92% increase. Coyote v2 reports kernel reconfiguration times of 72.3 ms for changing networking and numerical-kernel services (scenario #2), and 85.5 ms for removing a traffic sniffer while retaining RDMA (scenario #3) [20, Section 9.3, Table 3]. The corresponding estimated increases are 3.75% and 3.17%. These kernel times exclude disk I/O and the host buffer copy and have a narrower scope than the F3 invocation.

For a size-matched lower bound, dividing 34,162,708 bytes by Coyote v2's reported UltraScale+ ICAP rate of 800 MB/s gives approximately 42.70 ms [20, Section 5.3, Table 2]. The 2.713 ms inspection interval is 6.35% of that bound. The calculation excludes fixed PR overhead and does not establish sustained end-to-end reconfiguration performance for this bitstream.

These comparisons place the measured inspection cost in context without claiming a complete PR admission path. They use reference systems with different timing scopes, and the inspection interval itself includes host submission and input transport. Integrating the checker may change which transfers can be shared or overlapped. The measured result is that the resource-optimized checker produces a verdict in 1.118–4.394 ms for the real benchmark inputs while occupying a small fraction of the U55C's resources.

7 TODO: Discussion + Conclusion

References

[1] Thea Aarrestad, Vladimir Loncar, Nicolò Ghielmetti, Maurizio Pierini, Sioni Summers, Jennifer Ngadiuba, Christoffer Petersson, Hampus Linander, Yutaro Iiyama, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Dylan Rankin, Sergo Jindariani, Kevin Pedro, Nhan Tran, Mia Liu, Edward Kreinar, Zhenbin Wu, and Duc Hoang. 2021. Fast convolutional neural networks on FPGAs with hls4ml. *Machine Learning: Science and Technology* 2, 4 (2021), 045015. doi:10.1088/2632-2153/ac0ea1

[2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, 265–283.

[3] Christophe Bobda, Joel Mandebi Mbongue, Paul Chow, Mohammad Ewais, Naif Tarafdar, Juan Camilo Vega, Ken Eguro, Dirk Koch, Suranga Handagala, Miriam Leeser, Martin Herbordt, Hafsah Shahzad, Peter Hofste, Burkhard Ringlein, Jakub Szefer, Ahmed Sanaullah, and Russell Tessier. 2022. The Future of FPGA Acceleration in Datacenters and the Cloud. *ACM Transactions on Reconfigurable Technology and Systems* 15, 3 (2022), 1–42. doi:10.1145/3506713

[4] Andrew Boutros, Mathew Hall, Nicolas Papernot, and Vaughn Betz. 2020. Neighbors From Hell: Voltage Attacks Against Deep Learning Accelerators on Multi-Tenant FPGAs. In *2020 International Conference on Field-Programmable*

Technology (ICFPT). 103–111. doi:10.1109/ICFPT51103.2020.00023

[5] Adrian M. Caulfield, Eric S. Chung, Andrew Putnam, Hari Angepat, Jeremy Fowers, Michael Haselman, Stephen Heil, Matt Humphrey, Puneet Kaur, Joo-Young Kim, Daniel Lo, Todd Massengill, Kalin Ovtcharov, Michael Papamichael, Lisa Woods, Sitaram Lanka, Derek Chiou, and Doug Burger. 2016. A Cloud-Scale Acceleration Architecture. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*. 1–13. doi:10.1109/MICRO.2016.7783710

[6] Jayeeta Chaudhuri and Krishnendu Chakrabarty. 2022. Detection of Malicious FPGA Bitstreams using CNN-Based Learning. In *2022 IEEE European Test Symposium (ETS)*. 1–2. doi:10.1109/ETS54262.2022.9810438

[7] Jayeeta Chaudhuri and Krishnendu Chakrabarty. 2023. Detection and Classification of Malicious Bitstreams for FPGAs. In *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. doi:10.1145/3566097.3568346

[8] Javier Duarte, Song Han, Philip Harris, Sergo Jindariani, Edward Kreinar, Benjamin Kreis, Jennifer Ngadiuba, Maurizio Pierini, Ryan Rivera, Nhan Tran, and Zhenbin Wu. 2018. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation* 13, 07 (2018), P07027. doi:10.1088/1748-0221/13/07/P07027

[9] Rana Elnaggar, Jayeeta Chaudhuri, Ramesh Karri, and Krishnendu Chakrabarty. 2023. Learning Malicious Circuits in FPGA Bitstreams. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 3 (2023), 726–739. doi:10.1109/TCAD.2022.3190771

[10] Dennis R. E. Gnad, Fabian Oboril, and Mehdi B. Tahoori. 2017. Voltage Drop-based Fault Attacks on FPGAs Using Valid Bitstreams. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. 1–7. doi:10.23919/FPL.2017.8056840

[11] Claudionor N. Coelho Jr., Aki Kuusela, Shan Li, Hao Zhuang, Jennifer Ngadiuba, Thea Klaeboe Aarrestad, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, and Sioni Summers. 2021. Automatic Heterogeneous Quantization of Deep Neural Networks for Low-Latency Inference on the Edge for Particle Detectors. *Nature Machine Intelligence* 3 (2021), 675–686. doi:10.1038/s42256-021-00356-5

[12] Ron Kohavi. 1995. A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*. 1137–1145.

[13] Dario Korolija, Timothy Roscoe, and Gustavo Alonso. 2020. Do OS Abstractions Make Sense on FPGAs?. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 991–1010.

[14] Tuan La, Khoa Dang Pham, Joseph Powell, and Dirk Koch. 2021. Denial-of-Service on FPGA-based Cloud Infrastructures: Attack and Defense. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 3 (2021), 441–464. doi:10.46586/tches.v2021.i3.441-464

[15] Tuan Minh La, Kaspar Mätas, Nikola Grunchevski, Khoa Dang Pham, and Dirk Koch. 2020. FPGADefender: Malicious Self-Oscillator Scanning for Xilinx UltraScale+ FPGAs. *ACM Transactions on Reconfigurable Technology and Systems* 13, 3 (2020), 15:1–15:31. doi:10.1145/3402937

[16] Dina G Mahmoud, Ognjen Glamočanin, Francesco Regazzoni, and Mirjana Stojilović. 2023. Practical Implementations of Remote Power Side-Channel and Fault-Injection Attacks on Multitenant FPGAs. In *Security of FPGA-Accelerated Cloud Computing Environments*. Springer, 101–135.

[17] Charalampos Mainas, Martin Lambeck, Bruno Scheufler, Laurent Bindschaedler, Atsushi Koshiba, and Pramod Bhatotia. 2025. F3: An FPGA-accelerated FaaS Framework. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. doi:10.1145/3731545.3731582

[18] Alessandro Palumbo, Luca Cassano, Biagio Luzzi, Juan Antonio Hernández, Pedro Reviriego, Giuseppe Bianchi, and Marco Ottavi. 2022. Is your FPGA bitstream Hardware Trojan-free? Machine learning can provide an answer. *Journal of Systems Architecture* 128 (2022), 102543. doi:10.1016/j.sysarc.2022.102543

[19] George Provelengios, Daniel Holcomb, and Russell Tessier. 2020. Power Wasting Circuits for Cloud FPGA Attacks. In *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*.

[20] Benjamin Ramhorst, Dario Korolija, Maximilian Jakob Heer, Jonas Dann, Luhao Liu, and Gustavo Alonso. 2025. Coyote v2: Raising the Level of Abstraction for Data Center FPGAs. In *Proceedings of the ACM Symposium on Operating Systems Principles*. arXiv:2504.21538 [cs.AR] doi:10.1145/3731569.3764845

[21] Jens Rettkowski, Safdar Mahmood, Arij Shallufa, Michael Hübner, and Diana Göhringer. 2019. Inspection of Partial Bitstreams for FPGAs Using Artificial Neural Networks. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 83–86. doi:10.1109/IPDPSW.2019.00023

[22] Rye Stahle-Smith, Carter Antley, Jason D. Bakos, and Rasha Karakchi. 2026. Hardware-Accelerated Line-Rate Bitstream Screening for Secure FPGA Reconfiguration. arXiv:2605.08984 [cs.CR] https://arxiv.org/abs/2605.08984

[23] Rye Stahle-Smith and Rasha Karakchi. 2025. Real-time ML-based Defense Against Malicious Payload in Reconfigurable Embedded Systems. arXiv:2509.02387 [cs.CR] https://arxiv.org/abs/2509.02387

[24] Junghwan Yoon, Yeseul Seo, Jihun Jang, Minsoo Cho, Jaeseung Kim, Hyung-Min Kim, and Taesoo Kwon. 2018. A Bitstream Reverse Engineering Tool for FPGA Hardware Trojan Detection. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 2318–2320. doi:10.1145/3243734.3278487

Received TODO; revised TODO; accepted TODO